

OpenMP Programs on Data Sharing, Loop Scheduling, and Synchronization**Objectives**

At the end of this laboratory session, students will be able to:

1. Understand the concept of **data-sharing attributes** in OpenMP.
2. Apply appropriate data-sharing clauses such as:
 - shared
 - private
 - firstprivate
 - lastprivate
 - default
3. Understand synchronization issues such as **race conditions** in parallel programs.
4. Implement synchronization constructs including:
 - critical
 - atomic
 - barrier
 - reduction
5. Analyze and compare different **loop scheduling strategies**:
 - Static
 - Dynamic
 - Guided

Introduction

In a parallel program, multiple threads execute simultaneously and often access common data. Improper management of shared variables may result in **race conditions**, leading to incorrect program output. OpenMP provides **data-sharing clauses** and **synchronization constructs** to control how variables are accessed and updated by multiple threads.

OpenMP also provides several **loop scheduling policies** that determine how iterations of a parallel loop are distributed among threads. Choosing an appropriate scheduling strategy can significantly improve load balancing and overall performance.

1. Data Sharing Clauses**i. Shared**

A shared variable has only **one copy**, accessible by all threads.

```
#pragma omp parallel shared(x)
```

All threads access the same variable.

ii. Private

Each thread gets its own independent copy of the variable.

`#pragma omp parallel private(i)`

Changes made by one thread are not visible to others.

iii. Firstprivate

Each thread gets its own copy initialized with the original value.

`#pragma omp parallel firstprivate(a)`

Useful when every thread needs the initial value.

iv. Lastprivate

After loop execution, the value computed during the last iteration is copied back to the original variable.

`#pragma omp parallel for lastprivate(x)`

v. Default

Specifies the default data-sharing attribute.

`#pragma omp parallel default(shared)`

or

`#pragma omp parallel default(none)`

Using `default(none)` forces every variable to be explicitly declared as shared or private.

2. Synchronization Constructs

Synchronization prevents multiple threads from simultaneously modifying shared data.

OpenMP provides:

Critical Section

`#pragma omp critical`

Only one thread executes the block at a time.

Atomic

`#pragma omp atomic`

Protects a single memory update.

Barrier

`#pragma omp barrier`

All threads wait until every thread reaches the barrier.

Reduction

`#pragma omp parallel for reduction(+:sum)`

Combines partial results from all threads safely.

3. Loop Scheduling

OpenMP supports different scheduling policies for distributing loop iterations.

Static Scheduling

Iterations are divided equally among the threads.

`#pragma omp parallel for schedule(static)`

Dynamic Scheduling

Iterations are assigned dynamically as threads become free.

#pragma omp parallel for schedule(dynamic)

Guided Scheduling

Large chunks are assigned initially, with chunk sizes decreasing over time.

#pragma omp parallel for schedule(guided)

Exercises

1. **Implement an OpenMP program** to compute the **sum of the first N natural numbers** in parallel. Using the same program, demonstrate the effect of the following OpenMP data-sharing clauses:
 - shared
 - private
 - firstprivate
 - lastprivate

For each case:

1. Execute the program.
2. Observe the output produced.
3. Explain the effect of the data-sharing clause on:
 - Variable values
 - Data accessibility among threads
 - Correctness of the final result

Finally, compare the outputs obtained for each data-sharing clause and explain why they differ.

2. **Implement a C program using OpenMP** to process an integer array in parallel and demonstrate the use of the following synchronization constructs:
 - critical
 - atomic
 - reduction
 - master

The program should perform the following tasks:

1. The **master** thread should initialize the array and display the total number of threads participating in the computation.
2. Use the **reduction** clause to compute the sum of all array elements in parallel.
3. Use the **atomic** construct to count the number of even elements in the array.
4. Use the **critical** construct to allow each thread to safely display its thread ID and the partial sum of the array elements processed by that thread.
5. After all threads complete execution, the **master** thread should display the final sum of the array elements and the total count of even numbers.

3. **Implement an OpenMP program using C** to determine the number of prime numbers within a given range (1 to **N**) using the parallel for directive. Since checking whether a number is prime requires a varying amount of computation depending on the number being tested, this problem exhibits an imbalanced workload across loop iterations.

Execute the program separately using the following OpenMP scheduling policies:

- `schedule(static)`
- `schedule(dynamic)`
- `schedule(guided)`

For each scheduling policy:

- Record the thread ID responsible for processing each number.
- Measure the execution time using `omp_get_wtime()`.
- Observe how loop iterations are distributed among the threads.
- Compare the load balancing and execution time obtained with each scheduling policy.

4. Write a parallel program using OpenMP to implement the **Merge Sort** algorithm.

Analyze the performance by computing:

- Sequential execution time
- Parallel execution time
- Speedup
- Efficiency

for varying input sizes and thread counts.

Additional Exercises

1. Implement a **multithreaded tokenizer** using OpenMP. Assign the lines of a text file to threads in a round-robin fashion. Each thread should print:
 - Thread ID
 - Tokens extracted from its assigned lines.
2. Implement a parallel OpenMP program to generate the **histogram** of a given array. Compare the execution time of the sequential and parallel implementations.
3. Implement an OpenMP program to compute the factorial of **N** numbers using:
 - Critical Section
 - Atomic
 - Reduction
 - Master
 - Locks

Compare the execution time of each synchronization construct.

4. Implement a parallel OpenMP program to estimate the value of π using the **Monte Carlo (Random Shooting)** technique.

Hint

Randomly generate points inside the square $[0, 1] \times [0, 1]$.

Count the number of points inside the quarter circle.

Estimate π using

$$\pi = 4 \times \frac{\text{Number of points inside the circle}}{\text{Total number of generated points}}$$

Compare the execution time for different numbers of threads.